

MIS Diagrams

UML图型结构、数据输入与作用总览

一、快速对照表

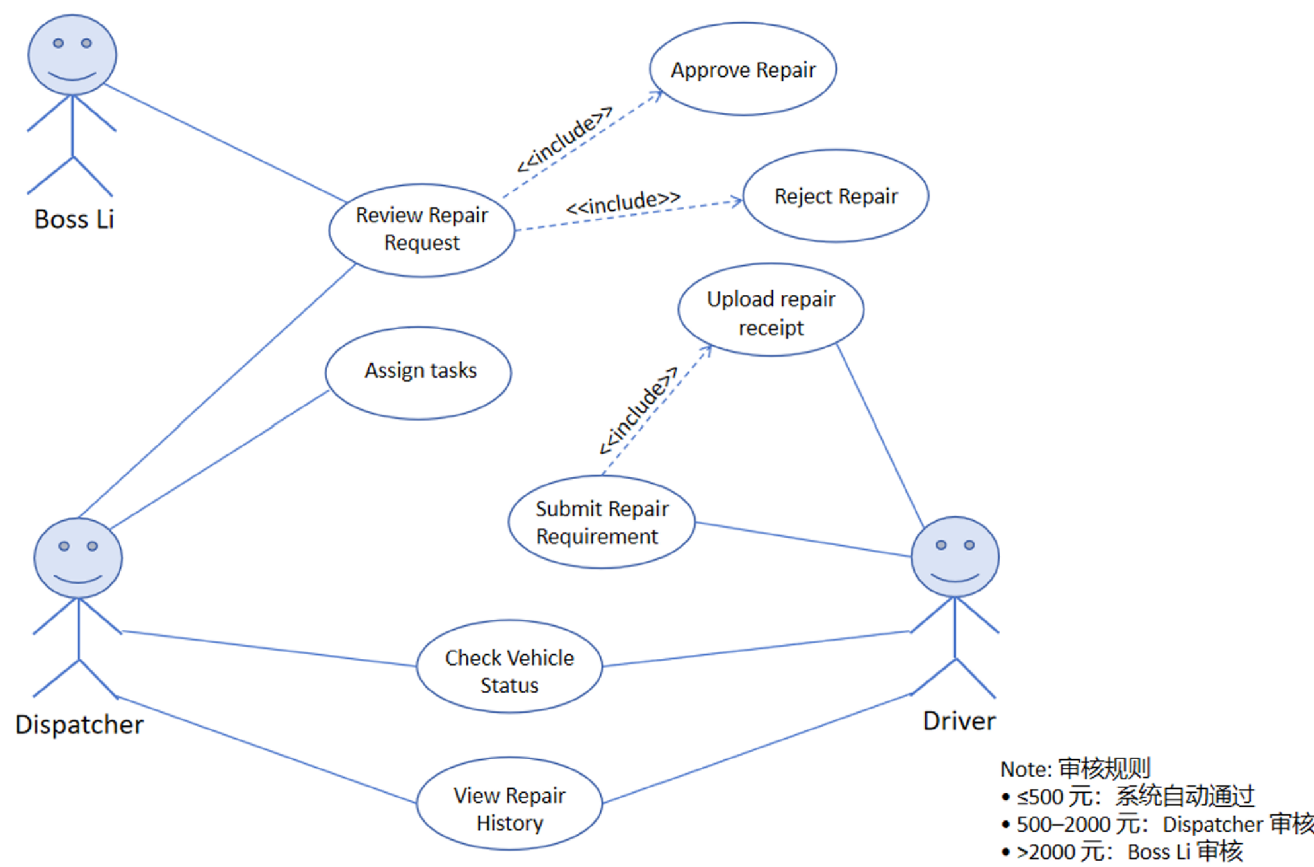
图型	输入数据	核心结构	输出/作用
用例图	功能需求列表、参与者角色	参与者 + 用例 + 系统边界 + 关系	功能范围界定
用例描述	单个用例的详细规格	表格：流程+前置/后置+异常	需求规格说明
活动图	业务流程步骤、决策点	泳道 + 活动节点 + 决策菱形	操作路径可视化
类图（核心）	领域实体、属性、关联关系	类框 + 多重性 + 关系线	静态结构定义
类图（三层）	架构分层、层间依赖	三层堆叠 + 类归属分层	技术架构设计
序列图	交互场景、消息顺序	生命线 + 激活条 + 消息箭头	动态交互建模
状态图	对象生命周期、事件转换	状态框 + 转换箭头 + 守卫条件	状态机设计
追溯矩阵	模型元素+需求的对应关系	行列交叉 ✓	一致性验证

二、各图型详细拆解

1. 用例图（Use Case Diagram）

输入数据：

- 需求规格说明书中的功能列表
- 系统参与者角色（Actor）清单
- 功能之间的关系（包含/扩展/继承）



关系类型速查：

关系	符号	含义
关联	实线（参与者→用例）	参与者能使用该功能
《include》	虚线箭头（A→B）	A必须包含B， B被复用
《extend》	虚线箭头（A→B）	B是A的可选扩展
泛化	空心三角箭头（子→父）	继承关系

作用： 界定系统功能范围、明确使用者角色、展示功能间依赖（考试常考：区分include和extend）

2. 用例描述（Use Case Description）

输入数据：

- 用例名称和编号
- 业务规则（BR）
- 非功能性需求（NF）

结构表格：

栏目	输入内容类型	示例
用例名称	功能名称 + 编号	UC02 提交维修报修申请
参与者	角色清单	主：司机；次：调度员
前置条件	执行前状态	司机已登录、车辆故障
主流程	正常路径步骤	1.打开→2.拍照→3.提交→4a/4b/4c分支
扩展流程	异常处理	E1无网络、E2照片无效
后置条件	执行后状态	RepairRequest已创建、Vehicle已更新
关联需求	需求追溯	BR01、BR02、NF01、NF02、NF03

作用： 补充用例图无法表达的流程细节，是活动图和序列图的直接输入源。

3. 活动图（Activity Diagram）

输入数据：

- 用例描述中的主流程和扩展流程
- 业务决策规则（BR）
- 组织角色分工

结构要素：

元素	图形	输入数据
泳道	纵向/横向分区	角色名称（司机/调度/系统）
开始节点	● 实心圆	无

元素	图形	输入数据
活动节点	圆角矩形	操作描述（“拍照上传”）
决策节点	◇ 菱形	判断条件（“金额<500?”）
同步条	粗黑线（并行分叉/汇合）	并行任务
结束节点	● 实心圆套空心圆	无

画法规范：

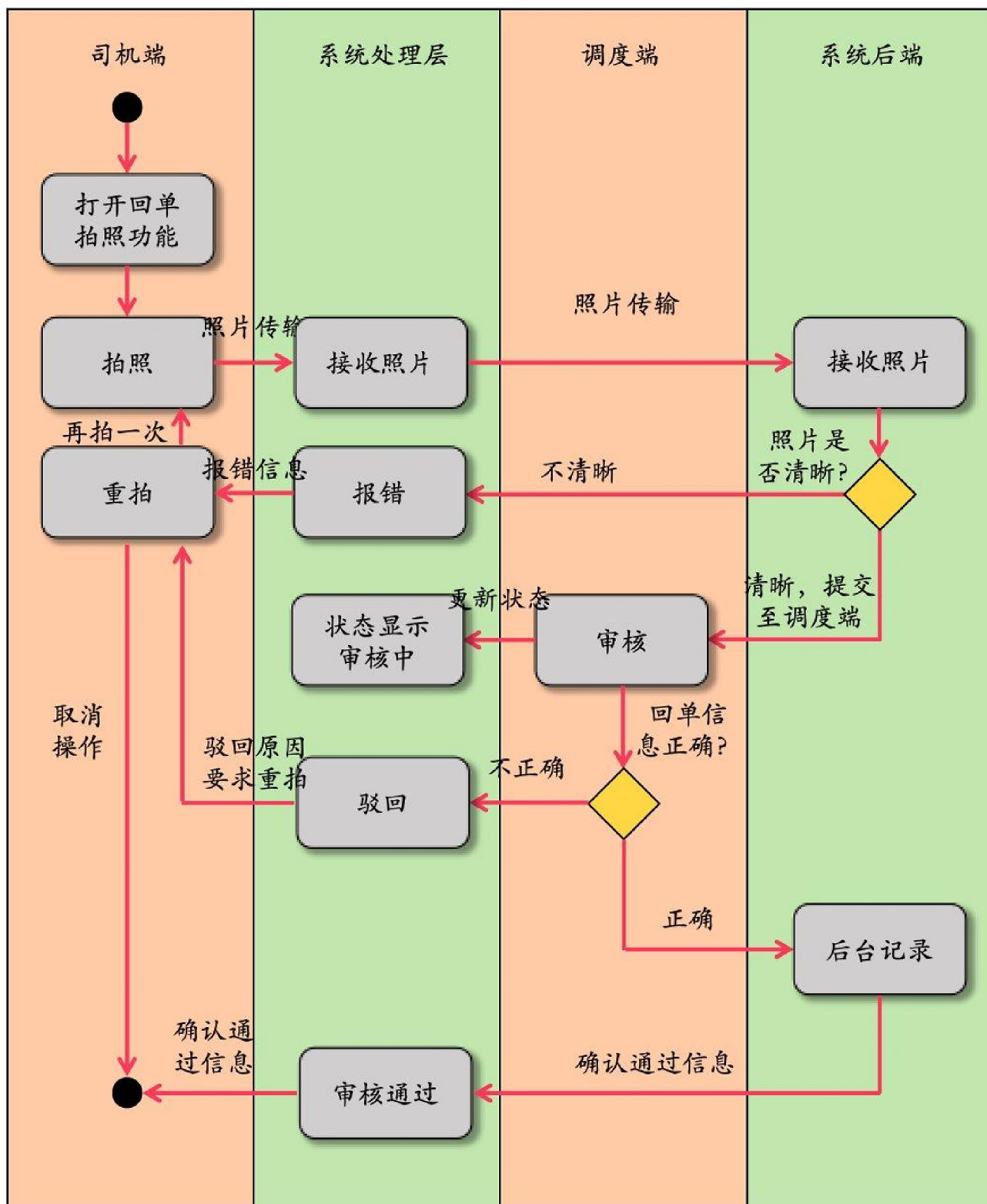


图 3-1 活动图一：回单拍照上传流程

作用： 业务流程可视化、识别并行/分支/循环、验证流程完整性（所有决策出口必须有路径）。

4. 类图 (Class Diagram)

4.1 核心类图 (领域模型)

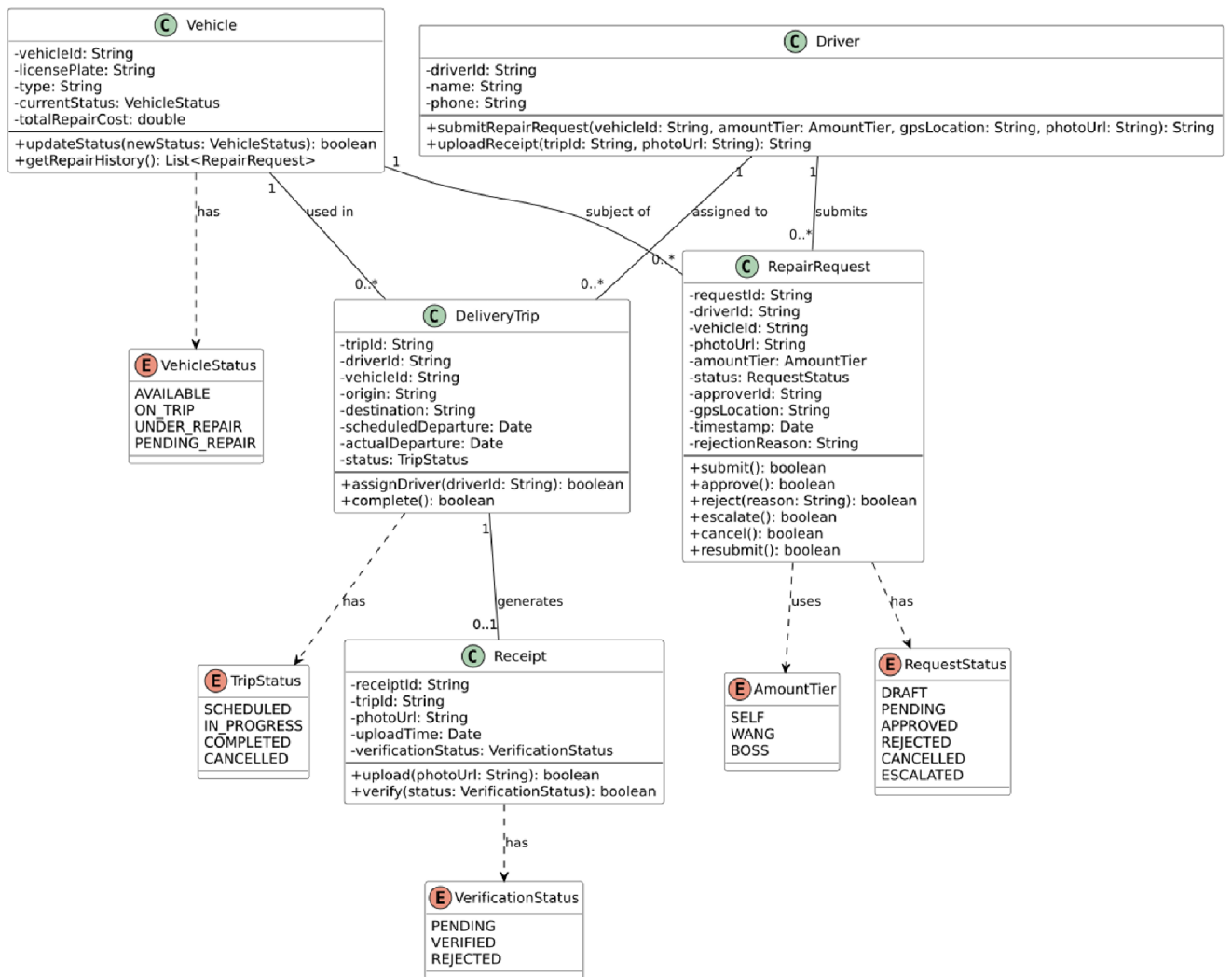
输入数据：

- 领域实体清单（从用例中提取）
- 实体属性（从需求中提取）
- 实体间关系（业务规则决定）
- 业务方法（从活动图/序列图归纳）

结构要素——类框：

Plain Text

类名 (斜体=抽象类)	← 第一格
- privateAttribute: Type + publicAttribute: Type # protectedAttribute: Type	← 第二格 (属性)
+ publicMethod(): ReturnType - privateMethod(): void	← 第三格 (方法)



结构要素——关系线：

关系	符号	输入依据
依赖	---->（虚线箭头）	一个类使用另一个类（临时）
关联	————（实线）	长期结构关系，标注多重性
聚合	——◇——（空心菱形）	整体-部分，部分可独立存在
组合	——◆——（实心菱形）	整体-部分，部分不可独立存在
泛化	——▷——（空心三角）	继承关系

多重性标注：

符号	含义
1	恰好一个

符号	含义
*	零到多个
0..1	零或一个
1..*	一个到多个
n...m	n到m个

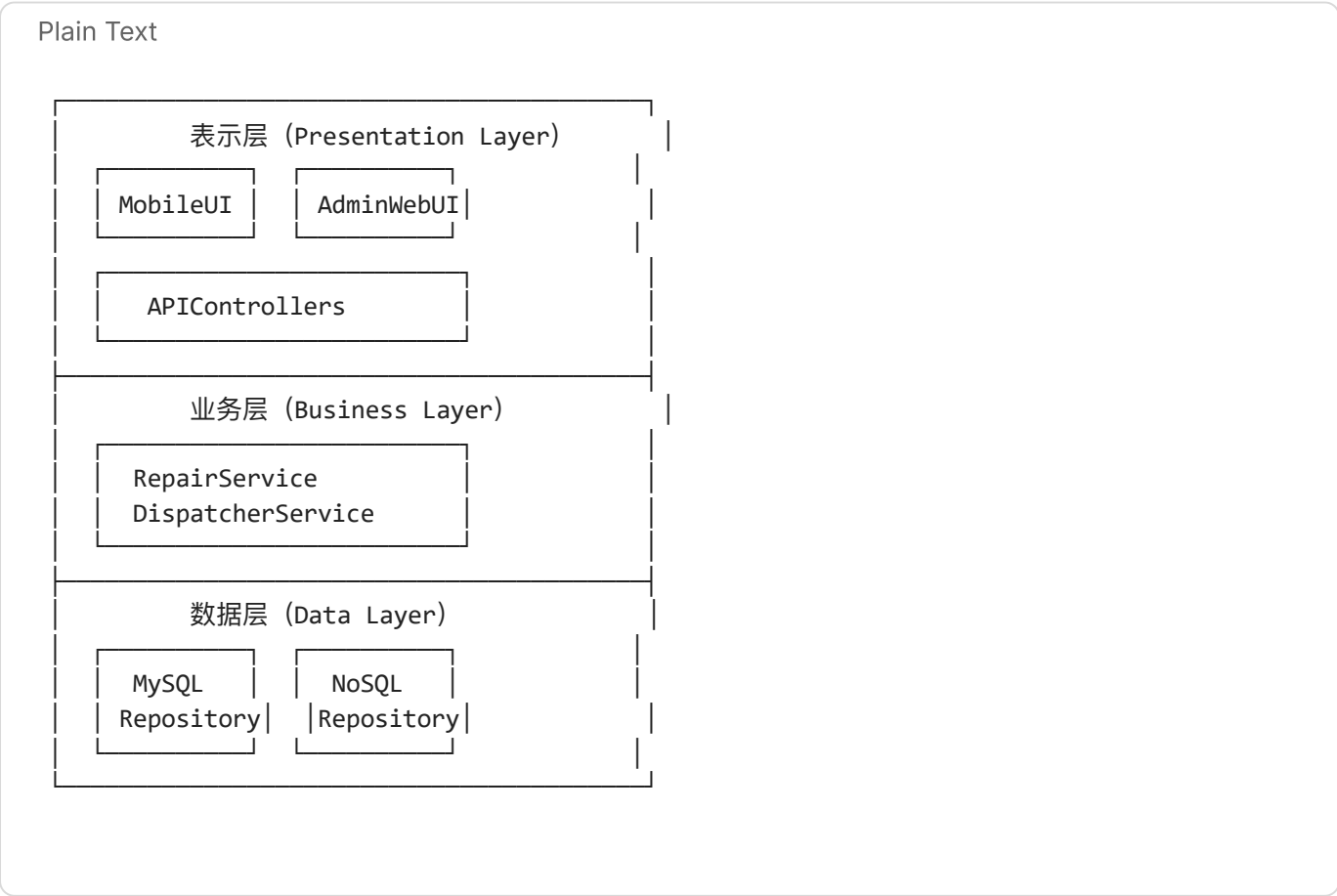
作用： 定义系统静态结构、确定数据持久化方案、为数据库设计提供依据。

4.2 三层架构类图

输入数据：

- 核心类图中的所有类
- 架构设计决策（分层策略）
- 技术选型（数据库类型、框架）

结构：



分层依赖规则：

- 上层可依赖下层，反之不可
- 同层可相互调用
- Controller只做路由，不含业务逻辑
- Service封装业务规则（BR）
- Repository封装数据访问

作用： 技术架构设计、指导代码组织、便于后续维护和替换。

5. 序列图（Sequence Diagram）

输入数据：

- 用例描述中的主流程
- 类图中的类和方法
- 交互场景（谁→谁→谁）

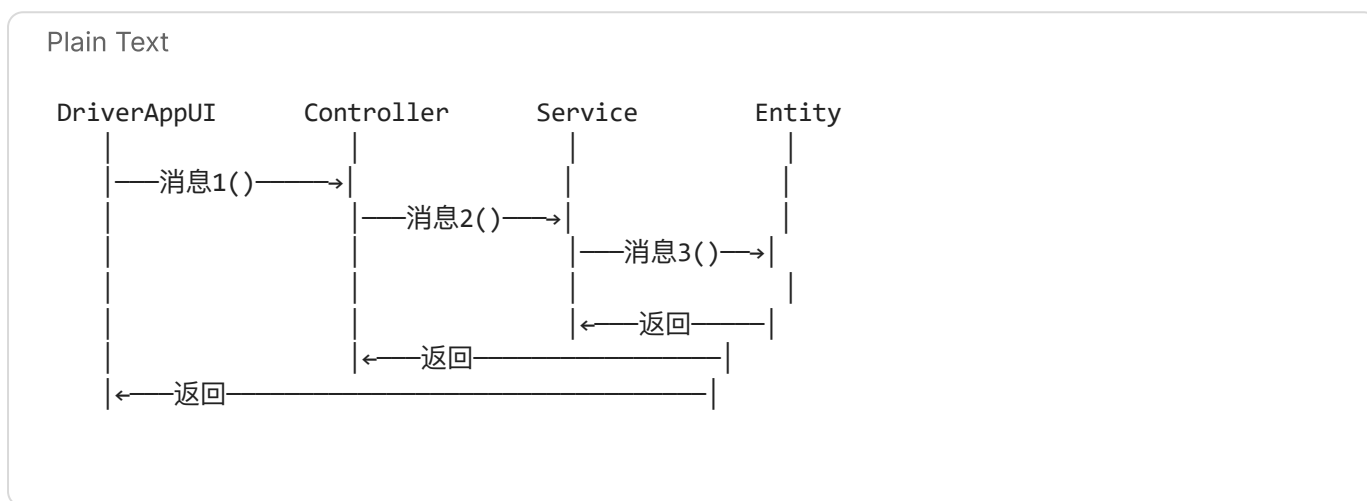
结构要素：

元素	画法	说明
生命线	—————┐ （对象名:类名）+ 向下虚线	每个参与者一条
激活条	■ （生命线上的细长矩形）	对象正在执行
同步消息	—————→ （实线箭头）	调用并等待返回
异步消息	—————→ （实线开放箭头）	调用不等待
返回消息	—— ———→ （虚线箭头）	返回值
自调用	——┐ 从自身发出回到自身	内部方法调用
组合片段	矩形框 + 左上角标注	alt/loop/opt/par

组合片段速查：

片段	含义	结构
alt	条件分支（互斥）	多条路径，用虚线分割
opt	可选执行	单条路径，带条件
loop	循环	标注循环次数或条件
par	并行	多条同时执行的路径

画法示例：



序列图1：司机提交维修报修申请（UC02）

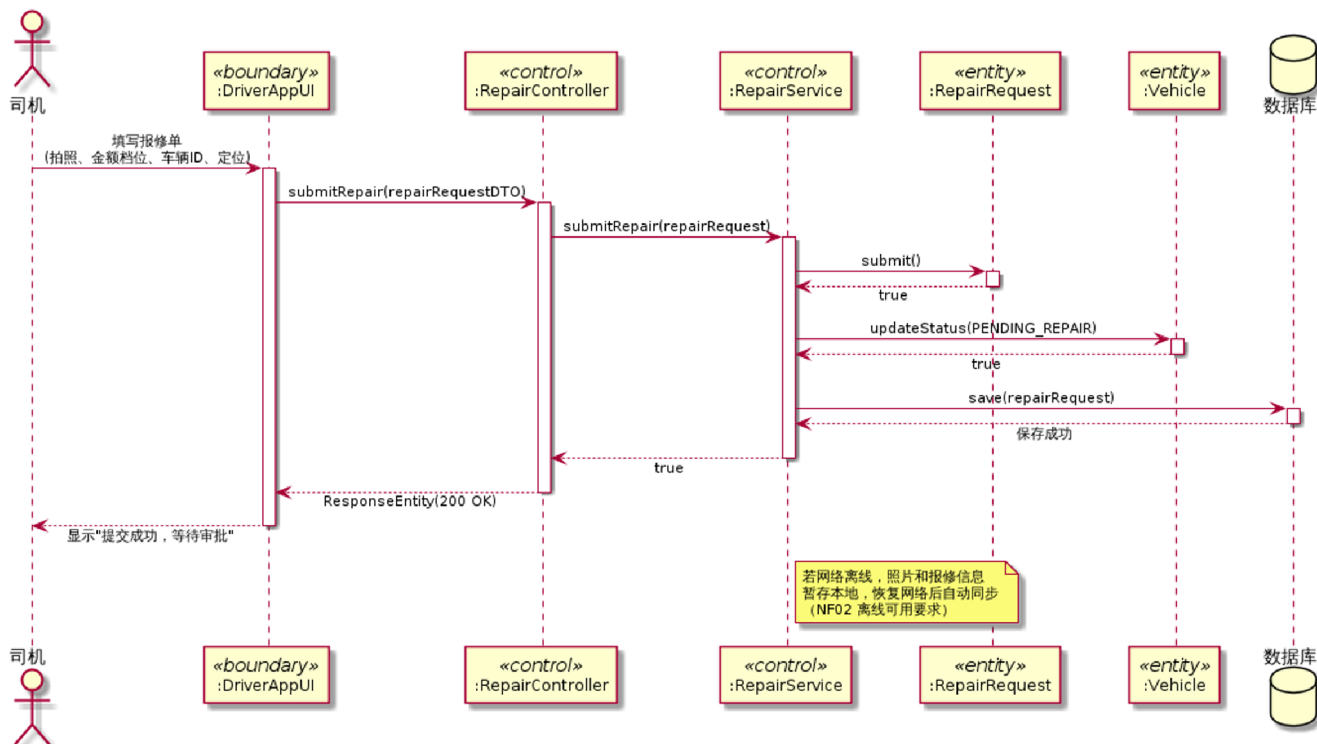


图 5-1 序列图一：司机提交维修报修申请

作用： 精确定义对象间交互顺序、验证方法调用是否在类图中存在、识别消息链是否符合分层架构。

6. 状态图（Statechart Diagram）

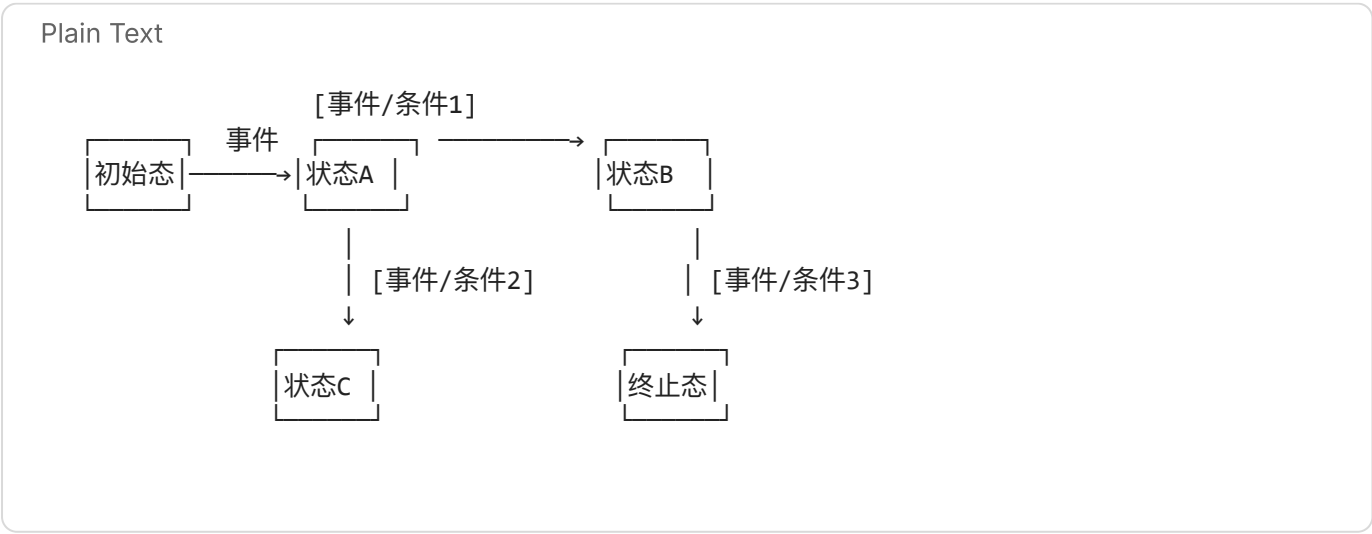
输入数据：

- 对象的生命周期定义
- 状态清单（从用例/活动图提取）
- 状态转换事件和守卫条件（BR）

结构要素：

元素	图形	输入数据
初始状态	● 实心圆	无
终止状态	◎ 实心圆套空心圆	无
状态	圆角矩形	状态名称（动词过去分词）
转换	箭头	[守卫条件] / 事件(参数)
选择伪状态	◇ 菱形	条件分支

状态图基本结构：



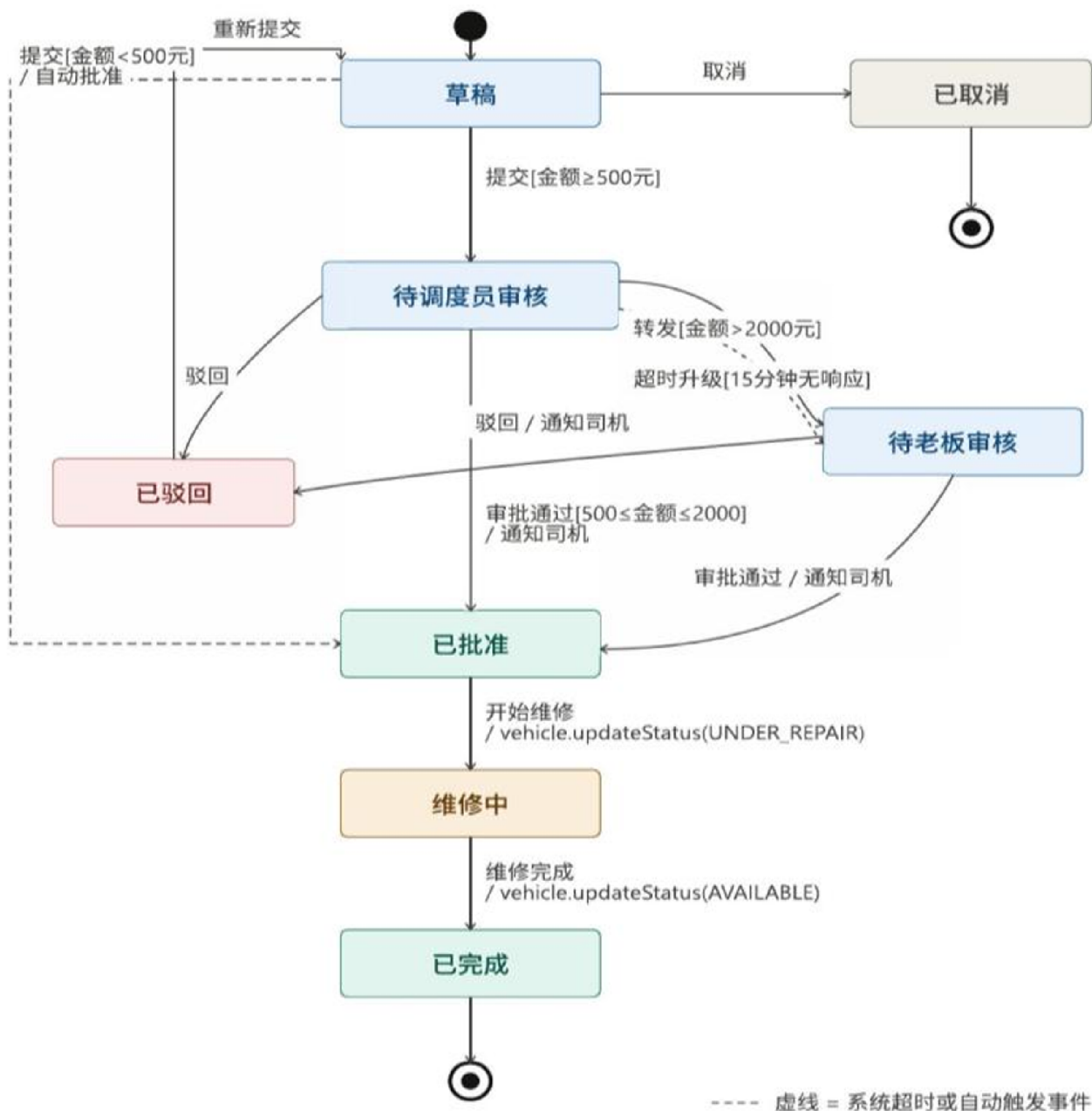


图 6-1 RepairRequest 状态图 (Statechart Diagram)

转换标注规范：

- 事件名(参数) —— 触发转换的操作
- [守卫条件] —— 必须满足的布尔表达式
- 事件[守卫]/动作 —— 完整格式

作用：建模对象生命周期、验证状态转换完整性（每个状态必须有入口和出口）、跨对象状态联动（如RepairRequest状态变化触发Vehicle状态变化）。

7. 追溯矩阵 (Traceability Matrix)

输入数据：

- 模型元素清单（类、方法、流程）
- 需求清单（UC、BR、NF）
- DFD流程编号

结构——类→需求矩阵：

类名	UC01	UC02	NF01	BR01	说明
Vehicle	✓	✓			状态变更贯穿全流程
RepairRequest		✓		✓	核心业务对象
Driver	✓	✓	✓		

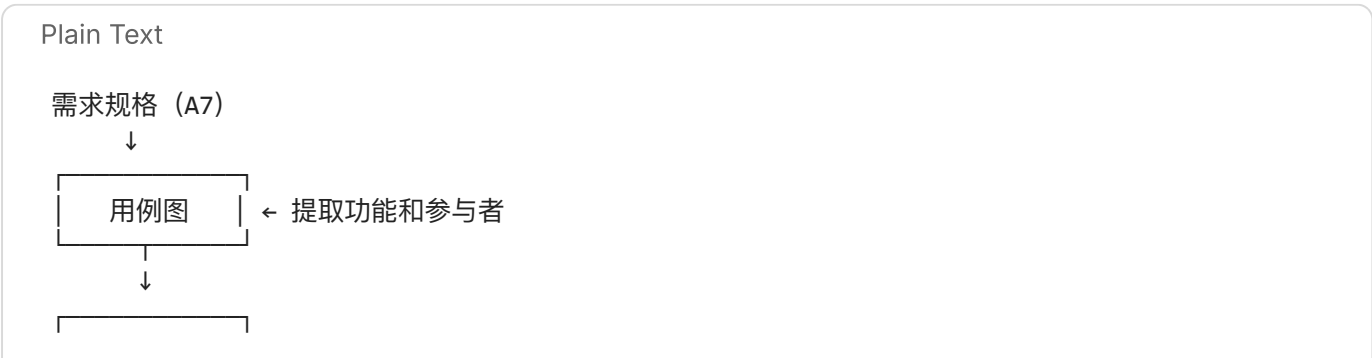
结构——方法一致性矩阵：

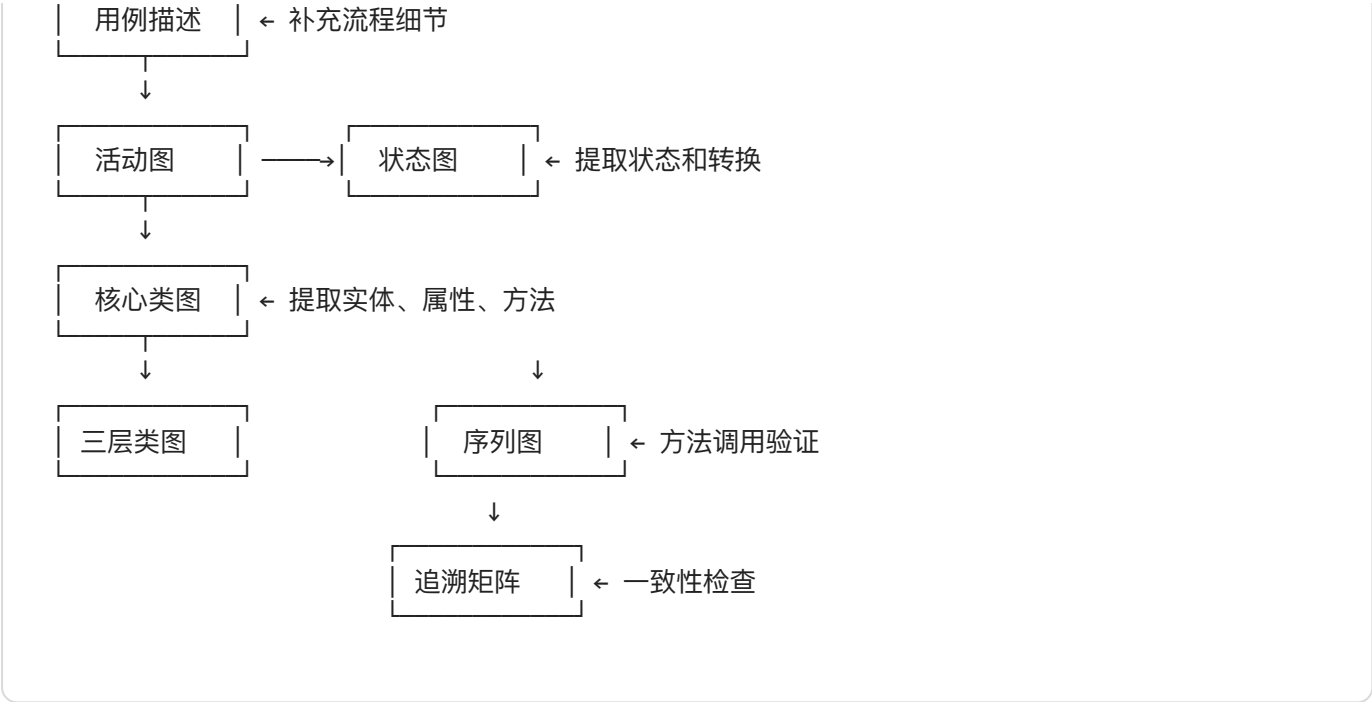
方法	所在类	调用方	类图定义	序列图使用
submit()	RepairRequest	RepairService	✓	✓
approve()	RepairRequest	RepairService	✓	✓

作用：

- 需求覆盖验证：每个需求是否有类实现
- 模型一致性验证：序列图中的每个方法是否在类图中定义
- 变更影响分析：需求变更时快速定位影响范围

三、图型间数据流关系





核心依赖链：用例描述 → 活动图 → 核心类图 → 序列图 → 追溯矩阵（方法一致性）

四、考试记忆卡片

各图型一句话功能：

图型	一句话功能
用例图	“系统给谁用、用什么功能”
用例描述	“功能具体怎么操作”
活动图	“流程怎么走、谁负责什么”
核心类图	“系统有什么数据实体”
三层类图	“代码怎么分层”
序列图	“对象间消息怎么传”
状态图	“对象生命周期怎么变”
追溯矩阵	“模型和需求是否一致”

各图型检查清单：

- 用例图：所有参与者都有关联？ include/extend使用正确？
- 活动图：每个决策节点所有出口都有路径？ 有开始和结束？
- 类图：多重性标注齐全？ 关系类型正确（关联/聚合/组合）？
- 序列图：消息接收方有对应方法？ alt分支覆盖所有条件？
- 状态图：每个状态都有入口和出口？ 守卫条件明确？
- 追溯矩阵：每个需求至少被一个类覆盖？ 序列图方法在类图中存在？